

Guardrails Specification

The safety and operational constraints that keep Meridian Pipeline Intelligence inside its intended scope — what each guardrail does, where it lives in the code, what failure it prevents, and how to demonstrate it live.

Meridian Pipeline Intelligence · ten guardrails in four layers · companion to the Presenter's Guide (ch. 5, 7, 8)

Governing principle — enforced by code, not by prompt. We never ask the model to behave; we make misbehavior either structurally impossible (there is no code path for it) or immediately visible (badges, flags, rejections shown on screen). The previous vendor's guardrails were instructions to the model — "do not make up numbers" — and they failed. None of the ten mechanisms below depends on the model following an instruction.

Layer 1 · Input guardrails — what questions get in

G1 Closed intent menu

question_router.py · llm_layer.py

The assistant can perform exactly eight pre-approved, hand-audited analyses. A question is matched to one of them — first by transparent keyword rules, then (optionally) by the LLM, whose entire routing authority is choosing one item from the fixed list. Its choice is re-validated against the catalog before use; anything else is discarded. There is no path from a user's words to a new computation.

PREVENTS Invented analyses — the "plausible query that's subtly wrong" failure of text-to-SQL approaches.

ENFORCED BY Whitelist check on the returned intent id; routing eval scored 22/24 on a golden set, with every miss failing to refusal, never to a wrong number.

DEMONSTRATE Ask "Are we going to make the number this quarter?" — the answer header reads "LLM classification, validated against the approved intent catalog."

G2 Out-of-scope refusal

question_router.py · _OUT_OF_SCOPE guard

Questions about periods or topics the data cannot support — Q3, next year, forecasts, predictions, anything non-pipeline — are refused before any other matching runs. The refusal is specific: it lists the analyses the tool does support. The LLM router is likewise instructed and validated to return "unsupported" rather than stretch.

PREVENTS Answering a question about the future with present-day figures; improvising when the honest answer is "no data exists."

DEMONSTRATE Ask "What will Q3 revenue be?" — refusal, with the supported-analyses list.

G3 Entity validation

llm_layer.py · _resolve_entity()

When a route names a person or segment, the name must resolve against the actual rep master (the ten real salespeople) or the fixed segment list. A misspelled, invented, or hallucinated entity resolves to nothing and the route is discarded. The model's output is never trusted as an identifier.

PREVENTS Reporting on a person who doesn't exist; attaching real numbers to a wrong name.

DEMONSTRATE Ask about "Bob Smith" — no such rep; the tool refuses rather than approximates.

Layer 2 · Computation guardrails — the LLM-free zone

G4 No model in the numeric path

analytics.py (entire module)

Every figure on screen is produced by deterministic pandas code — same inputs, same outputs, every time. The LLM cannot calculate, round, estimate, filter, pick a date range, or generate a query. This is not a rule the model follows; it is an architecture the model cannot violate: no code path carries model output into arithmetic.

PREVENTS Hallucinated numbers at the source — the failure that burned this client.

ENFORCED BY Module boundary: `analytics.py` imports no LLM library; 38 automated tests pin every important figure to hand-verified values.

G5 Fail-closed validation

data_loader.py · analytics.reconcile() · Dockerfile

On every launch the data must pass schema checks, safe parsing, and five independent reconciliations (deal-level totals must equal per-rep totals must equal per-segment totals; every row must land in exactly one bucket; the same-point subset must be contained in the final book). Any failure → the app serves nothing at all, with a specific error. The same gate runs inside the Docker build: an image whose data or tests fail cannot be produced.

PREVENTS Serving numbers from corrupted, drifted, or partially-loaded data; deploying a broken build.

DEMONSTRATE Sidebar: "Data validated · 5 reconciliations passed." (Or corrupt a CSV copy and show the full-stop error.)

Layer 3 · Output guardrails — what reaches the screen

G6 Numeric validator with veto power

llm_layer.py · narrate_answer() · extract_numbers()

The LLM may rephrase a computed answer into friendly prose. Before that prose renders, plain code extracts every number from it and checks each one against the set of numbers the engine actually produced. One mismatch — including abbreviations like "\$3.65M" for \$3,650,000, and silent rounding — and the entire narration is discarded, replaced by the engine's own wording, with the rejection displayed on screen. A made-up figure structurally cannot render.

PREVENTS Hallucinated or distorted numbers at the last mile — the narration step, which is the only place the model touches an answer.

ENFORCED BY Tests assert the validator rejects \$999,999, rejects 42.7%, rejects "\$3.65M"; the pass/fail is printed under every answer ("AI summary · 11 numbers verified").

G7 Self-downgrading trust badges

analytics.py · _trust_level() + per-answer rules

Every answer grades itself by fixed published rules — green (no known issue touches it), amber (exact arithmetic, but a stated assumption or data issue affects reading), red (a data defect dominates). Examples of the rules: booked revenue >50% re-dated (F4) → red; a rep's pipeline >50% on recycled IDs (F1) → the tool refuses to classify the person at all. The tool constrains its own confidence, not just the model's.

PREVENTS Technically-correct numbers being read as trustworthy when the underlying records are compromised; unfair conclusions about people.

DEMONSTRATE "Enterprise vs quota" (red — its whole numerator is re-dated) and "How is Priya Patel doing?" (red — 61% corrupted pipeline).

G8 Mandatory receipts

analytics.py · AnswerPacket contract

The only output shape the system has is an AnswerPacket: answer + source rows + written-out calculation + assumptions + warnings + trust level. An answer cannot exist without its evidence attached — traceability is not a feature to enable, it is the data structure itself.

PREVENTS "Based on your data" answers that can't be checked; arguments about where a number came from.

DEMONSTRATE Any answer → Source data tab. Enterprise is the showpiece: the numerator is one visible row.

G9 Graceful degradation

llm_layer.py · every call wrapped; provider-agnostic

Any LLM failure — missing key, timeout, rate limit, refusal, truncated response, vendor outage — degrades silently to keyword routing and the engine's own deterministic wording. The AI is an enhancement, never a dependency; the sidebar toggle turns it off live. The same design makes the tool vendor-agnostic: it has run on two different providers with zero change to any trust property, because the guardrails live in our code, not in any vendor's model.

PREVENTS An AI outage becoming an app outage; vendor lock-in of the safety story; half-finished AI text reaching users (truncated responses are treated as failures).

DEMONSTRATE Flip "LLM routing & narration" off in the sidebar — everything still works.

G10 Audit log

observability.py · logs/queries.jsonl

Every question is appended to an audit log: timestamp, question, how it was routed, the answer's trust level, whether the AI summary passed validation and how many numbers were checked, and response time. Logging is designed to never break the app (it swallows its own errors — tested), and the log persists across container restarts via a mounted volume. Anything the assistant has ever said can be audited after the fact.

PREVENTS Unaccountable answers; "what did it tell whom?" being unanswerable in an incident review.

DEMONSTRATE Sidebar → Observability: live session stats and the recent-questions list.

Summary — guardrail × failure prevented × evidence

	GUARDRAIL	FAILURE IT PREVENTS	EVIDENCE IT WORKS
G1	Closed intent menu	Invented analyses	Routing eval 22/24; misses → refusal
G2	Out-of-scope refusal	Improvised answers	Live: "What will Q3 revenue be?"
G3	Entity validation	Wrong-person reporting	Resolution against real rep master
G4	LLM-free numeric path	Hallucinated numbers	Module boundary + 38 golden tests
G5	Fail-closed validation	Serving corrupted data	5 reconciliations; test-gated Docker build
G6	Numeric validator (veto)	Last-mile distortion	Rejection tests incl. "\$3.65M"; verdict shown on screen
G7	Self-downgrading badges	Misplaced confidence	Fixed rules; Enterprise & Priya go red today
G8	Mandatory receipts	Uncheckable answers	AnswerPacket is the only output type
G9	Graceful degradation	AI outage = app outage	Survived a live provider billing outage unchanged
G10	Audit log	Unaccountable answers	logs/queries.jsonl + 4 tests incl. never-raises

Residual risks, stated honestly: a misroute within the menu shows a clearly-labeled answer to the wrong question (visible, correctable — but possible); narration *wording* could mislead even with correct numbers (mitigated by showing the deterministic wording in the Calculation tab); and guardrails only cover defects we've conceived — F4 was found by human audit after all tests passed, which is why the audit posture, not the test count, is the durable protection.